



The Amazing Final Project: Lumen

ME 100: Electronics for the Internet of Things

Authors

Jonathan Nguyen

Yahany Francisco

Zander Vaux

Contents

1	Summary	2
2	Introduction	2
2.1	Problem Identification	2
2.2	Initial and Final Solution	2
3	Sensors & Actuators	3
3.1	Sensors	3
3.1.1	MAX30102 Heart-Rate Sensor	3
3.1.2	NEO-6M GPS Module	3
3.1.3	Ultrasonic Distance Sensor (HC-SR04)	4
3.1.4	OLED Display (SSD1306)	4
3.2	Actuators	4
3.2.1	Piezo Buzzer	4
3.2.2	DC Vibration Motor (3V)	4
3.3	System Integration	4
4	Schematics	5
4.1	Sender Schematic	5
4.2	Receiver Schematic	6
4.3	Vibration Motor Schematic	7
5	Diagram for Sender and Receiver Code	7
6	Sender Code	7
6.1	Sender Main	8
6.2	GPS Parser Code (GPS Current)	9
7	Receiver Code	12
7.1	Vibration Motor	12
7.2	OLED, Ultrasonic, Buzzer	13
8	Lumen App	16
8.1	Description	16
8.2	User Flow	16
8.3	Diagram for App Code	17
8.4	Backend API Code	17
8.5	Heart Rate Data Display	20
9	Conclusion	25
10	Team Photo	25

1 Summary

Project Lumen is a smartwatch-integrated self-reflection band designed to address the growing mental health crisis among college students and young adults. The device combines real-time heart rate monitoring with intentional emotional check-in prompts to encourage mindful self-reflection and emotional awareness in high-stress environments.

The core innovation lies in the device's ability to detect moments of elevated stress through biometric monitoring and deliver non-intrusive vibration alerts paired with a companion mobile application that helps users track emotional patterns over time. Unlike existing wearable that optimize for productivity and efficiency, Lumen prioritizes **emotional health and intentional self-awareness**, positioning itself in the rapidly expanding biometric wellness market.

2 Introduction

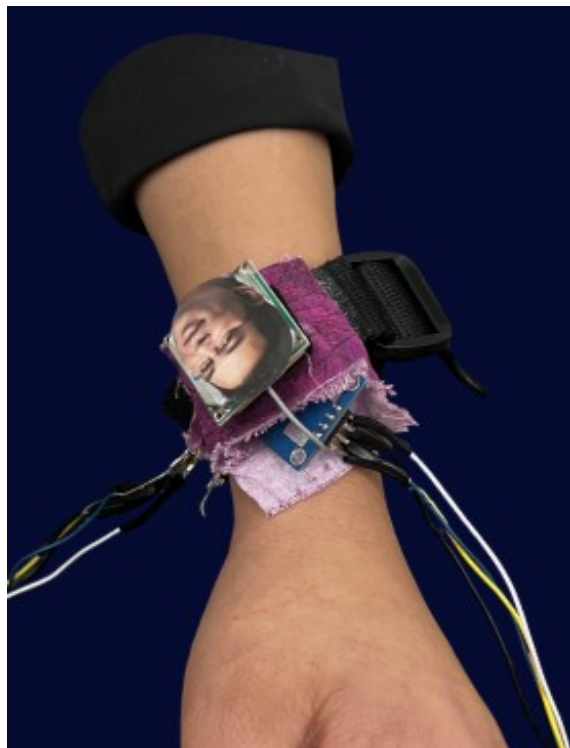
2.1 Problem Identification

In an age of constant connectivity and external demands, college students often find themselves detached from their own emotional and mental well-being. Although productivity tools such as study trackers and task organizers focus on maximizing efficiency, there is still a significant gap in promoting the reminder of how you are feeling by collecting health data from yourself anywhere you are right now, anytime.

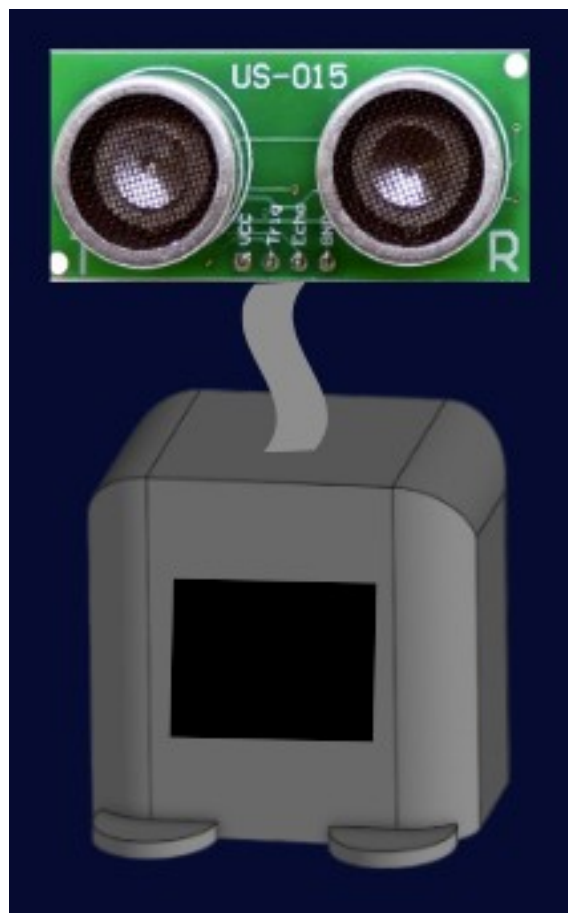
2.2 Initial and Final Solution

Originally, for the project, we proposed using a microphone sensor along with a heart rate sensor, GPS sensor, display screen, vibration motor, and buzzer. However, after our second design review, we realized that the microphone or voice recorder sensor might require more data and memory than the ESP32 could handle. Consequently, we decided to replace the voice recorder/microphone with an ultrasonic sensor. The main objective of the ultrasonic sensor is to wake up the display screen to show information regarding the beat-per-minute (BPM), spikes, and GPS location after the user inputs their data on the wristband.

This product consists of three integrated components. The wristband features a heart rate sensor and a Global Positioning System (GPS) sensor to track heart rate and current coordinates. It also includes a vibration motor for haptic feedback after user interaction. As data is gathered, a transmitter sends it to two receivers: the 'Buddy George' base station and the vibration motor on the wristband. The motor triggers a vibration whenever a heart rate spike occurs. The Buddy George unit stores heart rate data (BPM), spike records, and GPS coordinates (latitude and longitude), all timestamped in Pacific Standard Time. For data retrieval at home, users can trigger an ultrasonic sensor on a counter-top display by placing their hand near it to view the last recorded data from the wristband. The next page shows the two-component prototype of the product.



(a) Wristband Prototype



(b) Buddy George

3 Sensors & Actuators

This project incorporates a suite of sensors and actuators that together enable physiological monitoring, spatial awareness, user interaction, and guided reflection. Each component was selected for its compatibility with the ESP32, low power requirements, and ability to support a wearable wellness-focused system.

3.1 Sensors

3.1.1 MAX30102 Heart-Rate Sensor

The MAX30102 optical heart-rate sensor uses infrared and red LEDs paired with a photodetector to measure blood-volume changes at the skin surface. It operates over the I²C interface and provides continuous heart-rate readings, from which the system derives spike events associated with stress or heightened activity. This sensor forms the biological core of the smartband.

3.1.2 NEO-6M GPS Module

The NEO-6M GPS receiver provides geolocation data, enabling each heart-rate spike to be associated with a physical location. The smartband uses a lightweight GPS parser to extract latitude and longitude, supporting reflection features such as location-based journaling and pattern visualization in the companion app.

3.1.3 Ultrasonic Distance Sensor (HC-SR04)

Mounted on the reflection hub, the ultrasonic sensor detects when a user approaches the device. When the measured distance falls below a threshold, the system activates the display and begins the breathing guidance routine. This non-contact wake-up mechanism improves usability while conserving power.

3.1.4 OLED Display (SSD1306)

A 128×64 OLED screen is used in the reflection hub to show heart-rate summaries, spike counts, and GPS-derived locations. Its low power consumption and high contrast make it ideal for quick-glance feedback.

3.2 Actuators

3.2.1 Piezo Buzzer

A piezoelectric buzzer generates audio cues for the guided breathing cycle. Different tones represent inhale and exhale phases, while silent intervals mark hold and rest phases. PWM control from the ESP32 enables precise timing and frequency shaping to support mindfulness routines.

3.2.2 DC Vibration Motor (3V)

A compact DC vibration motor embedded in the smartband provides haptic feedback whenever a new spike is detected. Each spike corresponds to a vibration pulse, giving the wearer immediate, discreet awareness of stress-related physiological changes. A dedicated ESP32 drives the motor to avoid electrical interference with the heart-rate and GPS sensors.

3.3 System Integration

Together, these sensors and actuators create a multimodal system capable of:

- monitoring heart-rate signals,
- detecting and logging spike events,
- capturing associated GPS locations,
- awakening the reflection hub upon user approach,
- guiding breathing through audio cues,
- and delivering haptic notifications in real time.

This integrated design enables both real-time physiological awareness and structured end-of-day reflection through the companion mobile application.

4 Schematics

4.1 Sender Schematic

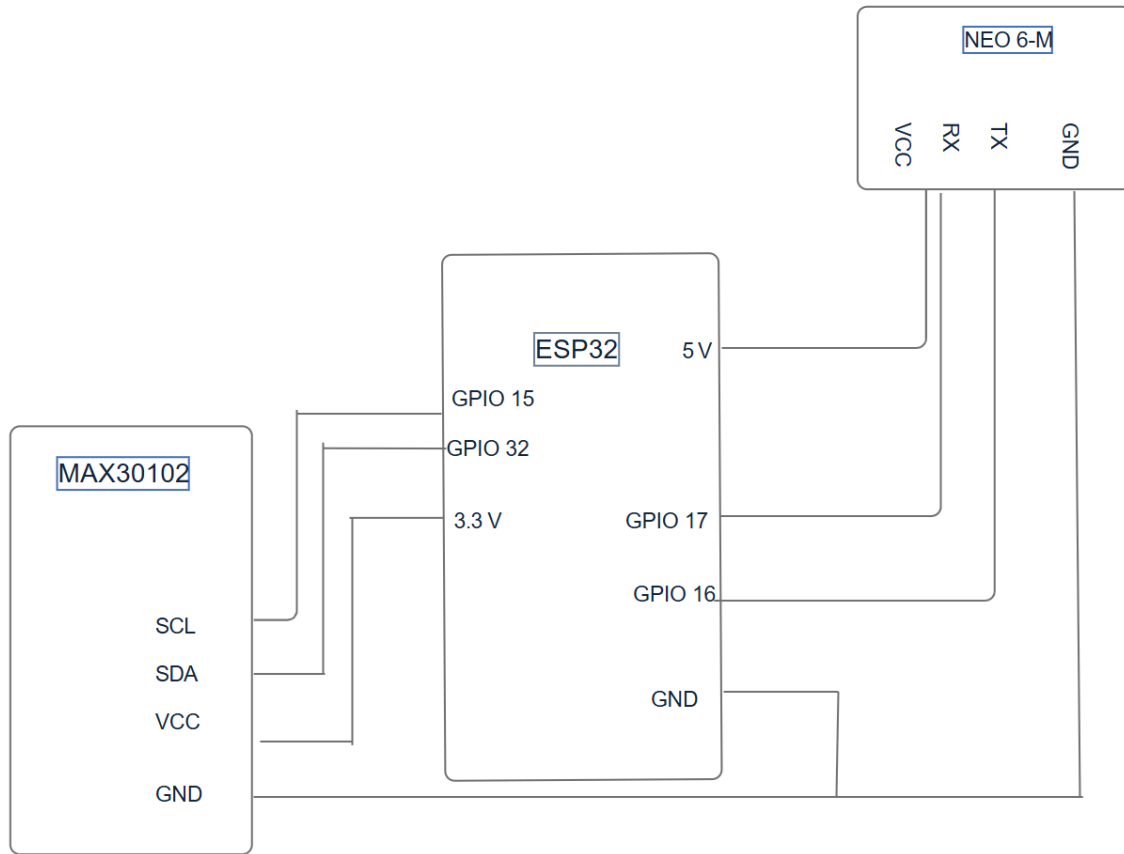


Figure 2: Hardware wiring diagram for the smartband system, showing connections between the ESP32, MAX30102 heart-rate sensor, and NEO-6M GPS module used for physiological sensing and geolocation tracking

4.2 Receiver Schematic

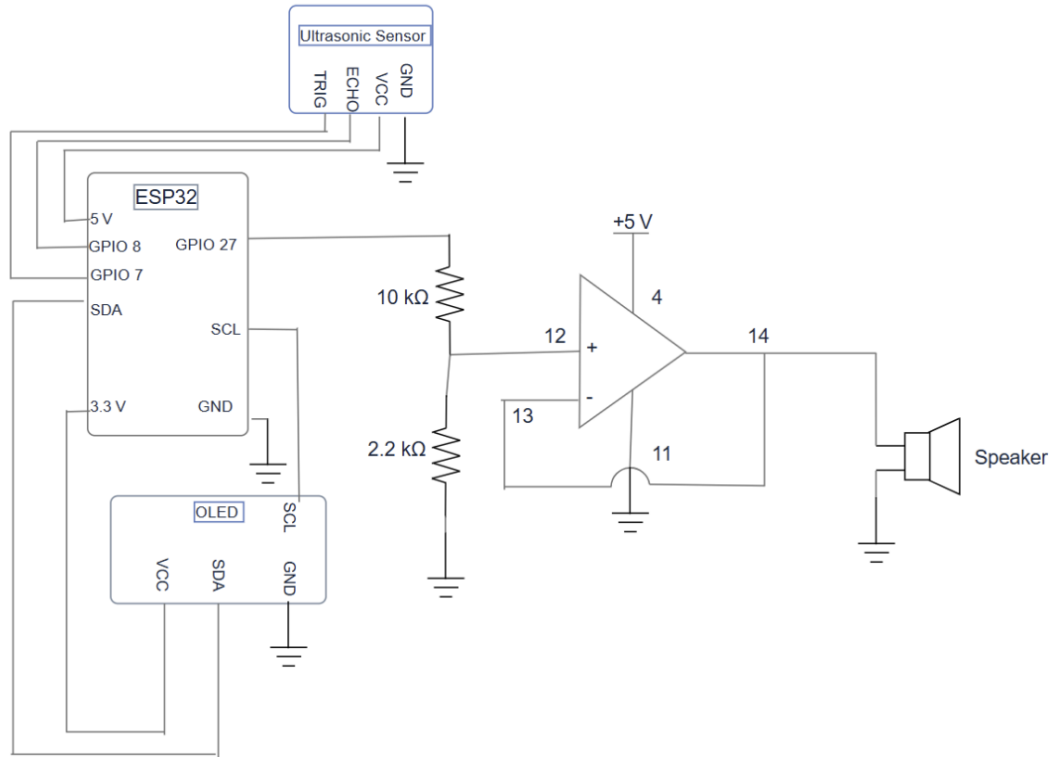


Figure 3: Receiver hardware schematic showing ESP32 connections to the ultrasonic sensor, OLED display, and amplified speaker used for breathing feedback

4.3 Vibration Motor Schematic

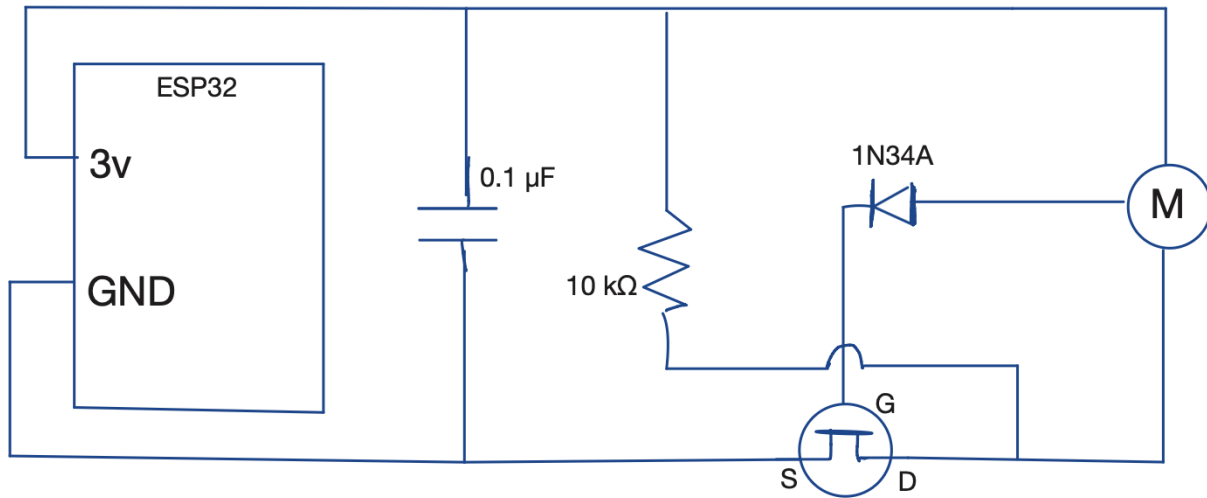


Figure 4: Vibration motor circuit using a series resistor, smoothing capacitor, and protection diode to safely interface the 3 V DC motor with the ESP32

5 Diagram for Sender and Receiver Code

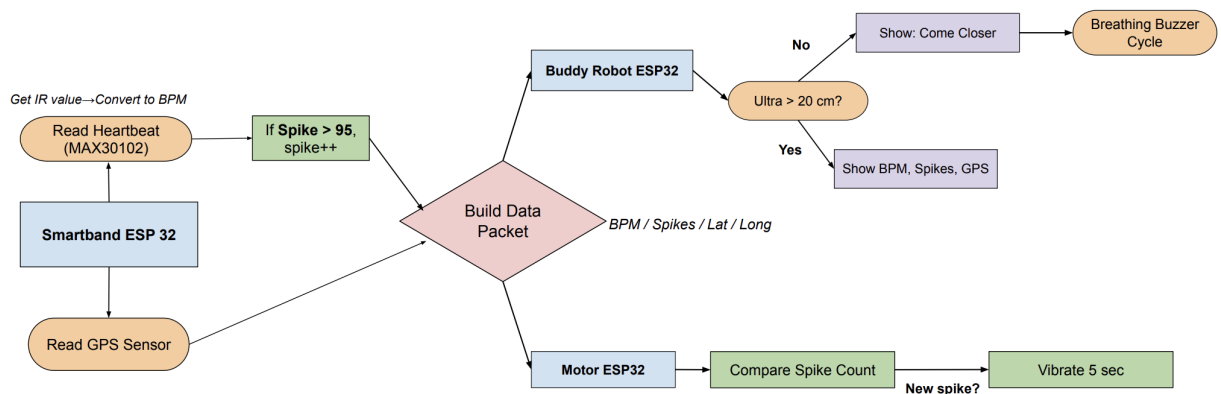


Figure 5: Code flow diagram for the smartband sender and buddy robot receiver, showing heartbeat sensing, spike detection, data packaging, and feedback actuation

6 Sender Code

The sender code collects data from the heart rate and GPS sensors to monitor BPM, spikes, and coordinates within the system. Once the code transmits this data to the two receivers, they trigger the actuators to allow access to the information.

6.1 Sender Main

```

1      #Code to send heartbeat data, record spikes, GPS, & communicate to vibration motor
        and OLED screen
2
3      import network, espnow
4      from machine import I2C, Pin
5      import time
6
7      from max30102 import MAX30102
8      from GPS_Current import get_current_latlon    # <-- Using GPS Parser
9
10     # Heartbeat Sensor
11
12     i2c = I2C(0, scl=Pin(15), sda=Pin(32), freq=400000)
13     sensor = MAX30102(i2c=i2c)
14     sensor.setup_sensor()
15     print("MAX30102 READY")
16
17     def get_bpm():
18         ir = sensor.get_ir()
19         if ir < 5000:
20             return 0
21         bpm = int((ir - 12000) / 80 + 70)
22         return max(0, min(bpm, 200))
23
24     # Spike Recording
25     #Counts each distinct time the BPM rises above the threshold, ignoring any subsequent
        high readings until the value first drops back down to "reset" the system
26
27     SPIKE_THRESHOLD = 95
28     spikes = 0
29     above = False
30
31     def update_spikes(bpm):
32         global spikes, above
33         if bpm > SPIKE_THRESHOLD and not above:
34             spikes += 1
35             above = True
36         if bpm <= SPIKE_THRESHOLD:
37             above = False
38         return spikes
39
40
41     SCREEN_MAC = b'\x14+\xafc\x00'    # OLED Screen MacAdress
42     MOTOR_MAC = b'\x14+\xae\xe4\xc4'  # Motor MacAdress
43
44     sta = network.WLAN(network.STA_IF)
45     sta.active(True)
46     sta.disconnect()
47     sta.config(channel=1)
48
49     e = espnow.ESPNow()
50     e.active(True)
51
52     e.add_peer(SCREEN_MAC)

```

```

53     e.add_peer(MOTOR_MAC)
54
55     print("SENDER READY")
56
57     while True:
58         bpm = get_bpm()
59         spike_count = update_spikes(bpm)
60
61         # ---- GPS code ----
62         gps_data = get_current_latlon()
63         if gps_data:
64             lat, lon = gps_data
65             gps_text = f"{lat:.5f}|{lon:.5f}"
66         else:
67             gps_text = "NO_FIX|NO_FIX"
68
69         print("BPM:", bpm, "Spikes:", spike_count, "GPS:", gps_text)
70
71         # ---- PACKET ----
72         packet = f"{bpm}|{spike_count}|{gps_text}"
73
74         try:
75             e.send(SCREEN_MAC, packet)
76             e.send(MOTOR_MAC, packet)
77         except:
78             print("ERROR SENDING")
79
80         time.sleep(0.2)
81
82

```

6.2 GPS Parser Code (GPS Current)

```

1     from machine import UART
2     import time
3     import gc
4
5     uart = UART(2, baudrate=9600, rx=7, tx=8, timeout=1000)
6     buff = bytearray(128)
7
8     UTC_OFFSET_HOURS = -8    # PST
9     current_fix = None
10    last_date_from_rmc = None
11
12    def dm_to_deg(dm):
13        if not dm or '.' not in dm or len(dm) < 4:
14            return None
15        try:
16            dot = dm.index('.')
17            deg = int(dm[:dot-2])
18            minutes = float(dm[dot-2:])
19            return deg + minutes / 60.0
20        except:
21            return None
22

```

```
23     #Parser
24
25     def parse_rmc(parts):
26         try:
27             if len(parts) < 10 or parts[2] != 'A':
28                 return None
29             t = parts[1]
30             d = parts[9]
31             lat_s, lat_h = parts[3], parts[4]
32             lon_s, lon_h = parts[5], parts[6]
33
34             if not (t and d and lat_s and lon_s):
35                 return None
36
37             hh, mm, ss = int(t[0:2]), int(t[2:4]), int(t[4:6])
38             dd, mo, yy = int(d[0:2]), int(d[2:4]), int(d[4:6])
39             year = 2000 + yy
40
41             lat = dm_to_deg(lat_s)
42             lon = dm_to_deg(lon_s)
43
44             if lat is None or lon is None:
45                 return None
46
47             if lat_h == 'S': lat = -lat
48             if lon_h == 'W': lon = -lon
49
50             return year, mo, dd, hh, mm, ss, lat, lon
51         except:
52             return None
53
54     def parse_gga(parts):
55         try:
56             if len(parts) < 7 or parts[6] == '0':
57                 return None
58
59             lat = dm_to_deg(parts[2])
60             lon = dm_to_deg(parts[4])
61             if lat is None or lon is None:
62                 return None
63
64             if parts[3] == 'S': lat = -lat
65             if parts[5] == 'W': lon = -lon
66             return lat, lon
67         except:
68             return None
69
70     def parse_gll(parts):
71         try:
72             if len(parts) < 7 or parts[6] != 'A':
73                 return None
74
75             lat = dm_to_deg(parts[1])
76             lon = dm_to_deg(parts[3])
77
```

```
78         if lat is None or lon is None:
79             return None
80
81         if parts[2] == 'S': lat = -lat
82         if parts[4] == 'W': lon = -lon
83
84         return lat, lon
85     except:
86         return None
87
88     # GPS Fetch
89
90     def get_current_latlon():
91         """
92         RETURNS:
93             - (lat, lon) from real GPS fix
94             - fallback coordinates when indoors
95         """
96         global current_fix
97
98         gc.collect()
99         n = uart.readinto(buff)
100
101         if n:
102             try:
103                 chunk = buff[:n].decode("utf-8", "ignore")
104             except:
105                 chunk = ""
106
107             for raw in chunk.split("\n"):
108                 line = raw.strip()
109                 if not line.startswith("$GP"):
110                     continue
111
112                 parts = line.split(",")
113                 tag = parts[0]
114
115
116                 if tag.startswith("$GPRMC"):
117                     parsed = parse_rmc(parts)
118                     if parsed:
119                         _, _, _, _, _, _, lat, lon = parsed
120                         current_fix = (lat, lon)
121
122                 elif tag.startswith("$GPGLL"):
123                     parsed = parse_gll(parts)
124                     if parsed:
125                         lat, lon = parsed
126                         current_fix = (lat, lon)
127
128                 elif tag.startswith("$GPGLL"):
129                     parsed = parse_gll(parts)
130                     if parsed:
131                         lat, lon = parsed
132                         current_fix = (lat, lon)
```

```
133
134     if current_fix:
135         return current_fix
136
137     return (FALLBACK_LAT, FALLBACK_LON)
```

7 Receiver Code

The code below illustrates the receiver (vibration motor) functionality, which triggers a vibration whenever a heart rate spike is detected.

7.1 Vibration Motor

```
1     import espnow, network
2     from machine import Pin
3     import time
4
5     #Motor Setup
6     vibe = Pin(26, Pin.OUT)
7
8     def vibrate_once():
9         # Strong 5-second vibration
10        vibe.value(1)
11        time.sleep(5)
12        vibe.value(0)
13        time.sleep(0.2)
14
15        sta = network.WLAN(network.STA_IF)
16        sta.active(True)
17        e = espnow.ESPNow()
18        e.active(True)
19
20        print("MOTOR RECEIVER READY")
21
22        last_spikes = 0
23
24        while True:
25            host, msg = e.recv()
26
27            if not msg:
28                continue
29
30            try:
31                text = msg.decode()
32            except:
33
34                print("Decode error:", msg)
35                continue
36
37            parts = text.split("|")
38
39            if len(parts) < 2:
40                print("Malformed packet:", text)
41                continue
42
```

```

43     # We only care about spikes
44     try:
45         spikes = int(parts[1])
46     except:
47         print("Spike parse error:", text)
48         continue
49
50     print("Received spikes:", spikes)
51
52     # Vibrate only when spike count increases
53     if spikes > last_spikes:
54         vibrate_once()
55
56     last_spikes = spikes
57

```

7.2 OLED, Ultrasonic, Buzzer

The code below shows the receiver with the ultrasonic sensor, OLED screen, buzzer for the component Buddy George after it receives the data from the sender code. This code then sends this data to the backend app.

```

1     import urequests
2     import espnow, network
3     from machine import Pin, I2C, PWM
4     import time
5     from ssd1306 import SSD1306_I2C
6
7     #Forward receiver data to backend app
8     def forward_to_backend(bpm, spikes, lat, lon):
9         try:
10             payload = {
11                 "bpm": bpm,
12                 "spikes": spikes,
13                 "lat": lat,
14                 "lon": lon
15             }
16             url = "http://10.41.252.97:5173/api/esp32"
17
18             response = urequests.post(url, json=payload, timeout=5)
19             print(f"Backend response: {response.status_code}")
20             response.close()
21         except Exception as e:
22             print(f"Backend error: {e}")
23
24     # OLED Display setup (or move down the description a little bit)
25     i2c = I2C(0, scl=Pin(32), sda=Pin(15))
26     oled = SSD1306_I2C(128, 64, i2c)
27     def show(text):
28         oled.fill(0)
29         y = 20 # Start 20 pixels down instead of 0
30         for line in text.split("\n"):
31             oled.text(line, 0, y)
32             y += 12
33         oled.show()
34

```

```

35     # Calibrate the distance for ultrasonic sensor
36     trig = Pin(7, Pin.OUT)
37     echo = Pin(8, Pin.IN)
38     def distance():
39         trig.off()
40         time.sleep_us(5)
41         trig.on()
42         time.sleep_us(10)
43         trig.off()
44         while echo.value() == 0:
45             pass
46         t1 = time.ticks_us()
47         while echo.value() == 1:
48             pass
49         t2 = time.ticks_us()
50         return (time.ticks_diff(t2, t1) * 0.0343) / 2
51
52     # Buzzer noise for inhale and exhale cycle by time interval
53     BUZZ_PIN = 27
54     buzzer_pwm = PWM(Pin(BUZZ_PIN, Pin.OUT), freq=800, duty=0)
55     INHALE_FREQ = 800    # higher tone for inhale
56     EXHALE_FREQ = 550   # lower tone for exhale
57     DUTY_CYCLE = 900    # 0-1023, adjust for volume
58
59     def play_tone(freq, duration_ms):
60         """Play a tone at given frequency for duration_ms"""
61         buzzer_pwm.freq(freq)
62         buzzer_pwm.duty(DUTY_CYCLE)
63         time.sleep_ms(duration_ms)
64         buzzer_pwm.duty(0)    # silence
65
66     def breathing_cycle(counter):
67         """
68         Breathing pattern tied to counter:
69         - Phases cycle every 40 ticks (at 100ms per tick = 4 seconds per cycle)
70         - Inhale: 0-10 ticks (1 sec) ; play HIGH tone
71         - Hold: 10-20 ticks (1 sec) ; silence
72         - Exhale: 20-30 ticks (1 sec) ; play LOW tone
73         - Rest: 30-40 ticks (1 sec) ; silence
74         """
75         phase = counter % 40
76         if phase < 10:
77             # INHALE phase / continuous high tone
78             if phase == 0: # Start of inhale, play the tone
79                 play_tone(INHALE_FREQ, 900) # 900ms tone over ~1 sec interval
80         elif phase < 20:
81             # HOLD phase / silence (already holding breath mentally)
82             buzzer_pwm.duty(0)
83         elif phase < 30:
84             # EXHALE phase / continuous low tone
85             if phase == 20: # Start of exhale, play the tone
86                 play_tone(EXHALE_FREQ, 900) # 900ms tone over ~1 sec interval
87         else:
88             # REST phase / silence
89             buzzer_pwm.duty(0)

```

```
90
91 # ESP-NOW
92 sta = network.WLAN(network.STA_IF)
93 sta.active(True)
94 sta.config(channel=1)
95 e = espnow.ESPNow()
96 e.active(True)
97 print("RECEIVER READY")
98 print("BUZZER ACTIVE WITH TONES")
99 last_bpm = last_spikes = last_lat = last_lon = "--"
100 counter = 0
101
102 # Main loop for the whole receiver system
103 while True:
104     msg = e.recv(0)
105     if msg:
106         peer, raw = msg
107         if raw:
108             try:
109                 bpm, spikes, lat, lon = raw.decode().split("|")
110                 last_bpm = bpm
111                 last_spikes = spikes
112                 last_lat = lat
113                 last_lon = lon
114             except Exception as err:
115                 print("Decode error:", err)
116         dist = distance()
117         if dist > 20:
118             show("COME CLOSER\nTO SEE STATS")
119         else:
120             show(f"BPM: {last_bpm}\nSpikes: {last_spikes}\nLat: {last_lat}\nLon:
121                  {last_lon}")
122             breathing_cycle(counter)
123             counter = (counter + 1) % 10000
124             time.sleep_ms(100)
```

8 Lumen App

8.1 Description

The SmartBand app serves as the user-facing interface for the ESP32-based wearable, transforming raw biometric and GPS data into meaningful insights. The app displays heart-rate spikes, location-tagged stress events, and breathing guidance in a clean, calming UI. Beyond data visualization, the app functions as a digital reflection journal, allowing users to record thoughts, emotions, and contextual notes alongside logged physiological events. A built-in community support feature enables users to connect with trusted friends, share selective insights, and encourage one another through stressful moments. Together, these features help users build awareness, track emotional patterns, and strengthen well-being through both personal reflection and social connection.

8.2 User Flow

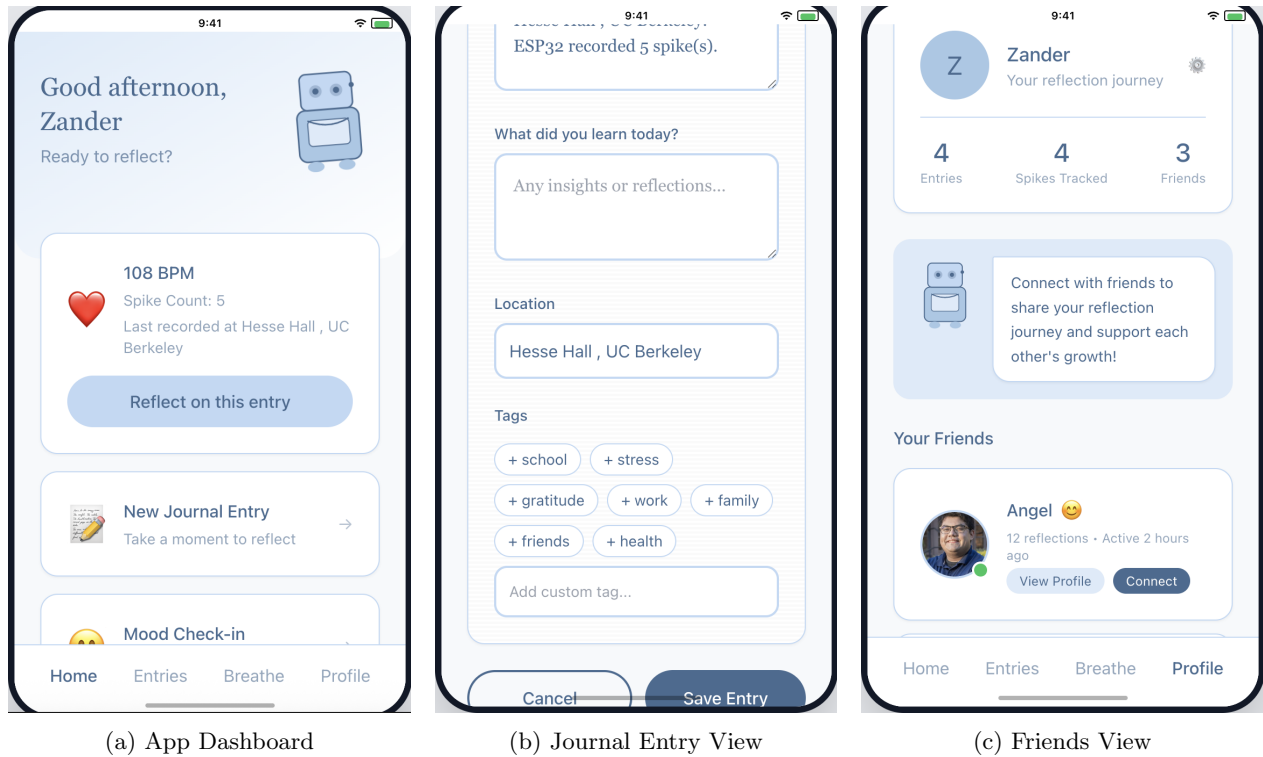


Figure 6: Application interface screenshots showing the dashboard, entry view, and friends view.

8.3 Diagram for App Code

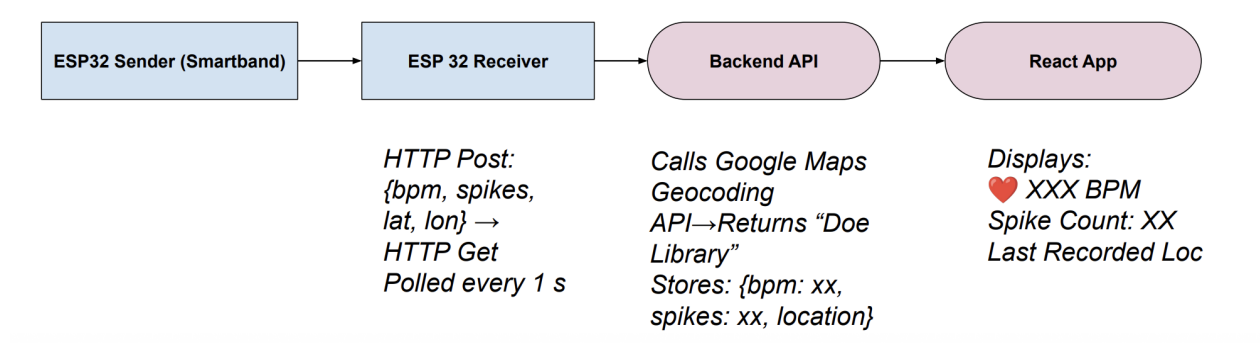


Figure 7: App-level data flow from the ESP32 receiver to the React interface via a backend API

8.4 Backend API Code

```

1  // Store latest ESP32 data in memory
2  let latestData = {
3    bpm: null,
4    spikes: null,
5    lat: null,
6    lon: null,
7    location: null,
8    timestamp: Date.now(),
9  };
10
11 // Google Maps Geocoding API configuration
12 // Get API key from environment variable or .env file
13 // Vite automatically loads .env files, but you can also set it as an environment
   variable
14 const GOOGLE_MAPS_API_KEY = process.env.GOOGLE_MAPS_API_KEY ||
   process.env.VITE_GOOGLE_MAPS_API_KEY || '';
15
16 // Convert coordinates to specific Berkeley campus location using Google Maps API
17 async function getLocationFromCoords(lat, lon) {
18   // Default location if coordinates are not provided
19   if (!lat || !lon || lat === '--' || lon === '--') {
20     return 'Location not available';
21   }
22
23   // If no API key, return basic location
24   if (!GOOGLE_MAPS_API_KEY) {
25     const latNum = parseFloat(lat);
26     const lonNum = parseFloat(lon);
27     if (latNum >= 37.8 && latNum <= 37.9 && lonNum >= -122.3 && lonNum <= -122.2) {
28       return 'Berkeley, CA';
29     }
30     return `Lat: ${lat}, Lon: ${lon}`;
31   }
32 }

```

```
33 try {
34   const latNum = parseFloat(lat);
35   const lonNum = parseFloat(lon);
36
37   // Use Google Maps Geocoding API
38   const url =
39     `https://maps.googleapis.com/maps/api/geocode/json?latlng=${latNum},${lonNum}&key=${GOOGLE_MAPS_KEY}`;
40   const response = await fetch(url);
41   const data = await response.json();
42
43   if (data.status === 'OK' && data.results && data.results.length > 0) {
44     // Find the most specific result (usually the first one)
45     const result = data.results[0];
46
47     // Extract building name or formatted address
48     // For Berkeley campus, look for specific building names
49     const formattedAddress = result.formatted_address;
50
51     // Try to extract a more specific location name
52     // Check for Berkeley campus buildings in address_components
53     let locationName = formattedAddress;
54
55     // Look for specific Berkeley landmarks/buildings
56     for (const component of result.address_components) {
57       // Check for building names or specific locations
58       if (component.types.includes('establishment') ||
59         component.types.includes('point_of_interest')) {
60         locationName = component.long_name;
61         break;
62       }
63     }
64
65     // If it's on UC Berkeley campus, try to get more specific
66     if (formattedAddress.includes('Berkeley') || formattedAddress.includes('UC
67       Berkeley')) {
68       // Use the formatted address, but try to extract building name
69       const buildingMatch = formattedAddress.match(/([^\s,]+),/);
70       if (buildingMatch) {
71         locationName = buildingMatch[1].trim();
72       }
73     }
74
75     return locationName || formattedAddress;
76   } else {
77     // Fallback to coordinates if geocoding fails
78     return `Berkeley, CA (${latNum.toFixed(4)}, ${lonNum.toFixed(4)})`;
79   }
80 } catch (error) {
81   console.error('Google Maps Geocoding error:', error);
82   // Fallback to basic location
83   const latNum = parseFloat(lat);
84   const lonNum = parseFloat(lon);
85   return `Berkeley, CA (${latNum.toFixed(4)}, ${lonNum.toFixed(4)})`;
```

```

86 }
87
88 export function esp32ApiPlugin() {
89   return {
90     name: 'esp32-api',
91     configureServer(server) {
92       // Parse JSON body for POST requests
93       server.middlewares.use('/api/esp32', async (req, res, next) => {
94         // Handle CORS
95         res.setHeader('Access-Control-Allow-Origin', '*');
96         res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
97         res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
98
99         if (req.method === 'OPTIONS') {
100           res.statusCode = 200;
101           res.end();
102           return;
103         }
104
105         if (req.method === 'POST') {
106           // Handle POST from ESP32 receiver
107           let body = '';
108           req.on('data', chunk => {
109             body += chunk.toString();
110           });
111
112           req.on('end', async () => {
113             try {
114               const data = JSON.parse(body);
115
116               // Get location from coordinates (async)
117               const location = await getLocationFromCoords(data.lat, data.lon);
118
119               // Update latest data
120               latestData = {
121                 bpm: data.bpm || null,
122                 spikes: data.spikes || null,
123                 lat: data.lat || null,
124                 lon: data.lon || null,
125                 location: location,
126                 timestamp: Date.now(),
127               };
128
129               res.setHeader('Content-Type', 'application/json');
130               res.statusCode = 200;
131               res.end(JSON.stringify({ status: 'ok' }));
132             } catch (error) {
133               console.error('Error parsing ESP32 data:', error);
134               res.statusCode = 400;
135               res.end(JSON.stringify({ error: 'Invalid JSON' }));
136             }
137           });
138         } else if (req.method === 'GET') {
139           // Handle GET from app frontend
140           res.setHeader('Content-Type', 'application/json');

```

```

141
142 // Return latest data or default sample data if no data received yet
143 if (latestData.bpm === null) {
144   // Return sample data for development
145   res.end(JSON.stringify({
146     bpm: 145,
147     spikes: 3,
148     location: "Doe Library, UC Berkeley",
149     lat: null,
150     lon: null,
151     timestamp: Date.now(),
152   }));
153 } else {
154   res.end(JSON.stringify(latestData));
155 }
156 } else {
157   next();
158 }
159 });
160 }
161 };
162 }

```

8.5 Heart Rate Data Display

```

1 import { motion } from 'framer-motion';
2 import { useState, useEffect } from 'react';
3 import Card from './Card';
4 import Button from './Button';
5
6 const HeartRateChart = ({ spikes, onSpikeClick, onESP32Click }) => {
7   const [esp32Data, setEsp32Data] = useState(null);
8   const [loading, setLoading] = useState(true);
9
10  useEffect(() => {
11    // Only fetch ESP32 data if there are no spikes
12    if (spikes && spikes.length > 0) {
13      setLoading(false);
14      return;
15    }
16
17    // Initial fetch
18    const fetchESP32Data = async () => {
19      try {
20        const response = await fetch('/api/esp32');
21        if (response.ok) {
22          const data = await response.json();
23          setEsp32Data(data);
24        }
25      } catch (error) {
26        console.error('Failed to fetch ESP32 data:', error);
27      } finally {
28        setLoading(false);
29      }
30    };

```

```

31
32     fetchESP32Data();
33
34     // Poll every second for real-time updates
35     const interval = setInterval(() => {
36         fetchESP32Data();
37     }, 1000);
38
39     return () => clearInterval(interval);
40 }, [spikes]);
41
42 if (!spikes || spikes.length === 0) {
43     if (loading) {
44         return (
45             <Card>
46                 <div className="text-center py-8">
47                     <p className="text-accent-navy opacity-60">Loading...</p>
48                 </div>
49             </Card>
50         );
51     }
52
53     if (esp32Data) {
54         return (
55             <Card>
56                 <div className="flex items-center gap-4 mb-4">
57                     <div className="text-4xl"></div>
58                     <div className="flex-1">
59                         <h3 className="font-medium text-accent-navy mb-1">
60                             {esp32Data.bpm} BPM
61                         </h3>
62                         <p className="text-sm text-accent-navy opacity-60">
63                             Spike Count: {esp32Data.spikes}
64                         </p>
65                         <p className="text-sm text-accent-navy opacity-60 mt-1">
66                             Last recorded at {esp32Data.location}
67                         </p>
68                     </div>
69                 </div>
70                 {onESP32Click && (
71                     <Button
72                         onClick={() => onESP32Click(esp32Data)}
73                         variant="secondary"
74                         className="w-full"
75                     >
76                         Reflect on this entry
77                     </Button>
78                 )}
79             </Card>
80         );
81     }
82
83     return (
84         <Card>
85             <div className="text-center py-8">

```

```

86     <p className="text-accent-navy opacity-60">No heart rate data yet</p>
87     <p className="text-sm text-accent-navy opacity-40 mt-2">
88       Your ESP32 will record spikes here
89     </p>
90   </div>
91 </Card>
92 );
93 }
94
95 // Find max BPM for scaling
96 const maxBPM = Math.max(...spikes.map((s) => s.bpm || 0));
97 const minBPM = Math.min(...spikes.map((s) => s.bpm || 0));
98 const range = maxBPM - minBPM || 100;
99 const baseline = minBPM;
100
101 // Normal BPM range: 60-100
102 const normalMax = 100;
103 const spikeThreshold = 120; // High spike threshold
104
105 const getSpikeColor = (bpm) => {
106   if (bpm >= spikeThreshold) return '#ef4444'; // Red for high spikes
107   if (bpm >= normalMax) return '#f59e0b'; // Orange for elevated
108   return '#466B91'; // Blue for normal
109 };
110
111 const getSpikeHeight = (bpm) => {
112   const normalized = ((bpm - baseline) / range) * 100;
113   return Math.max(20, Math.min(100, normalized));
114 };
115
116 return (
117   <Card>
118     <div className="mb-4">
119       <h3 className="font-medium text-accent-navy mb-1">Heart Rate Spikes</h3>
120       <p className="text-xs text-accent-navy opacity-60">
121         {spikes.length} spike{spikes.length !== 1 ? 's' : ''} detected today
122       </p>
123     </div>
124
125     <div className="relative">
126       {/* Chart area */}
127       <div className="h-48 relative border-b border-calm-blue-light">
128         {/* Normal range indicator */}
129         <div
130           className="absolute left-0 right-0 bg-calm-blue-lighter opacity-30"
131           style={{
132             top: `${100 - ((normalMax - baseline) / range) * 100}%`,
133             height: `${((normalMax - baseline) / range) * 100}%`,
134           }}
135         </div>
136
137       {/* Spikes */}
138       <div className="flex items-end justify-between h-full gap-2 px-2">
139         {spikes.map((spike, index) => {
140           const height = getSpikeHeight(spike.bpm);

```

```

141     const color = getSpikeColor(spike.bpm);
142     const isHigh = spike.bpm >= spikeThreshold;
143
144     return (
145       <motion.div
146         key={spike.id || index}
147         initial={{ height: 0 }}
148         animate={{ height: `${height}%` }}
149         transition={{ delay: index * 0.1, duration: 0.5 }}
150         onClick={() => onSpikeClick && onSpikeClick(spike)}
151         className={`flex-1 rounded-t-lg cursor-pointer hover:opacity-80
152           transition-opacity relative group ${
153             isHigh ? 'ring-2 ring-red-400' : ''
154           }`}
155         style={{ backgroundColor: color, minWidth: '40px' }}
156       >
157         {/* Tooltip */}
158         <div className="absolute -top-12 left-1/2 transform -translate-x-1/2
159           opacity-0 group-hover:opacity-100 transition-opacity
160           pointer-events-none whitespace-nowrap">
161           <div className="bg-accent-navy text-white text-xs px-2 py-1 rounded">
162             {spike.bpm} BPM
163             {spike.location && (
164               <div className="text-xs opacity-90 mt-1">{spike.location}</div>
165             )}
166           </div>
167         </div>
168       </motion.div>
169     );
170   }
171 }
172 </div>
173
174 {/* Legend */}
175 <div className="flex items-center justify-between mt-4 text-xs text-accent-navy
176   opacity-60">
177   <div className="flex items-center gap-2">
178     <div className="w-3 h-3 rounded bg-accent-navy" />
179     <span>Normal</span>
180   </div>
181   <div className="flex items-center gap-2">
182     <div className="w-3 h-3 rounded bg-amber-500" />
183     <span>Elevated</span>
184   </div>
185   <div className="flex items-center gap-2">
186     <div className="w-3 h-3 rounded bg-red-500" />
187     <span>High Spike</span>
188   </div>
189 </div>
190
191 {/* High spikes list */}
192 {spikes.filter((s) => s.bpm >= spikeThreshold).length > 0 && (
193   <div className="mt-4 pt-4 border-t border-calm-blue-light">
194     <p className="text-sm font-medium text-accent-navy mb-2">

```

```

192     High Spikes ({spikes.filter((s) => s.bpm >= spikeThreshold).length})
193   </p>
194   <div className="space-y-2">
195     {spikes
196       .filter((s) => s.bpm >= spikeThreshold)
197       .map((spike, index) => (
198         <motion.div
199           key={spike.id || index}
200           initial={{ opacity: 0, x: -10 }}
201           animate={{ opacity: 1, x: 0 }}
202           transition={{ delay: index * 0.1 }}
203           onClick={() => onSpikeClick && onSpikeClick(spike)}
204           className="flex items-center justify-between p-2 bg-red-50 rounded-lg
205             cursor-pointer hover:bg-red-100 transition-colors"
206         >
207           <div className="flex items-center gap-3">
208             <div className="text-2xl">|</div>
209             <div>
210               <p className="text-sm font-medium text-accent-navy">
211                 {spike.bpm} BPM
212               </p>
213               {spike.location && (
214                 <p className="text-xs text-accent-navy opacity-60">
215                   {spike.location}
216                 </p>
217               )}
218               {spike.timestamp && (
219                 <p className="text-xs text-accent-navy opacity-40">
220                   {new Date(spike.timestamp).toLocaleTimeString('en-US', {
221                     hour: '2-digit',
222                     minute: '2-digit',
223                   })}
224                 </p>
225               )}
226             </div>
227             <div>
228               <button className="text-accent-navy opacity-60 hover:opacity-100">
229                 Reflect
230               </button>
231             </div>
232           </div>
233         </motion.div>
234       )}
235     </div>
236   </Card>
237 );
238 export default HeartRateChart;

```

9 Conclusion

This project successfully integrated hardware, firmware, and software into a cohesive smart wearable system capable of monitoring physiological signals, detecting stress events, and presenting the data in a meaningful, user-centered way. By combining a MAX30102 heart-rate sensor, a GPS module, and dual ESP32 communication channels, the smartband reliably captures real-time biometric information and transmits it wirelessly to both a display module and a companion mobile application. The receivers translate this data into actionable feedback, including a breathing-guided buzzer response and visualized spike events, while the app extends functionality through location tagging, journaling, and community support features.

The final prototype demonstrates how embedded sensing, wireless communication, and thoughtful interface design can work together to support emotional well-being and self-reflection. Although future iterations could improve robustness, battery life, and enclosure design, the system establishes a solid foundation for a fully deployable wearable reflection tool. This project highlights the potential of accessible embedded systems to create meaningful human-centered health technologies and opens pathways for continued development in personal wellness monitoring.

10 Team Photo



Figure 8: On the left is Zander, middle is Yahany, and right is Jonathan

References

- [1] G. Anwar, “ME100 Lab 6,” *ME100: Electronics of IoT*, University of California, Berkeley, Fall 2025
- [2] Nelia. *MAX30102 MicroPython Driver*. GitHub repository: <https://github.com/n-elia/MAX30102-MicroPython-driver>. Accessed: December 5, 2025.